

SILICON DREAMS

Study Guide

MODULE 3 · WEEK 3 · CM-HW-101-M3

SG-M3-01 pairs with TB-M3-01 / TB-M3-02

Tape-out Environment Bootstrap

Clone mod3-starter-repo, pin LibreLane 2025.04, run the first hardening pass end-to-end.

Learning objectives

- Install LibreLane 2025.04 as a pinned container and confirm the SKY130A PDK downloads correctly.
- Pull mod3-starter-repo and locate top.v, arbiter.v, clock_gate.v, axiom_shim.v, and the LibreLane config.
- Run `librelane run` on the starter RTL and observe every flow stage emit a green tick.
- Recognise a healthy synthesis log vs a stage-crashed log; know which report directory holds which answer.

Prep checklist

- ☐ Module 2 fork is pushed, CI is green, rubric score recorded.
- ☐ Docker 24+ installed, `docker run hello-world` succeeds.
- ☐ At least 16 GB RAM and 30 GB free disk — LibreLane runs are not small.
- ☐ GitHub CLI authenticated. You will push a lot more frequently this module.
- ☐ Your ChipFoundry shuttle account is registered (you should have received an invite email after M2 sign-off).

Step 1 · Fork and clone mod3-starter-repo

Unlike M2, Module 3 is a NEW repo. You will not extend your M1/M2 fork — M3 restructures the project for multi-module integration.

```
gh repo fork chipfoundry/silicon-dreams-m3-starter --clone --remote
cd silicon-dreams-m3-starter
git checkout -b my-tapeout
```

Familiarise yourself with the directory layout:

src/top.v	Top wrapper. Wires elevator + arbiter + clock_gate + axiom_shim together.
src/elevator.v	Slot for your hardened M2 RTL. Paste your fault-tolerant M2 design here.
src/arbiter.v	NEW in M3. Round-robin with priority override. Has one deliberate bug — see SG-M3-03.
src/clock_gate.v	Thin ICG wrapper around sky130_fd_sc_hd__dlclkp. See SG-M3-04.
src/axiom_blackbox.v	The AXIOM black-box stub. Do NOT modify — it is instantiated by name and bound at implementation.
src/axiom_shim.v	Defensive wrapper around the black-box. See SG-M3-05.
librelane/config.json	Flow configuration. CLOCK_PERIOD, SYNTH_STRATEGY, PL_TARGET_DENSITY_PCT, etc.
librelane/pin_order.cfg	Pad-ring order — defines how top-level ports map to the tt_um pins.
test/	cocotb tests for arbiter, clock gate, axiom_shim. Integration smoke test in test/top/.
.github/workflows/	drc.yml, lvs.yml, timing.yml. Each PR must pass all three.

Step 2 · Pin LibreLane to 2025.04

Module 3 is reproducible only if the flow version is pinned. The course grader runs 2025.04 — newer versions may emit different DRC classifications.

```
# Pull the pinned image
docker pull efabless/librelane:2025.04

# Check the pin in your repo
cat librelane/config.json | grep version
# Expected: "meta.version": "2025.04"

# Run the shortcut the repo provides
./scripts/run-librelane.sh --version-check
```

If the version check fails, the grader will reject your submission even if hardening succeeds. Do not edit the pin.

Setting up LibreLane Environment

for linux setup

```
sudo apt update
sudo apt upgrade -y
```

```
sudo apt install -y \  
curl \  
git \  
wget \  
build-essential \  
python3 \  
python3-pip \  
python3-venv \  
python3-tk \  
make \  
unzip
```

Then install Nix (recommended):

```
cd ~
```

```
wget https://github.com/NixOS/nix-installer/releases/latest/download/nix-installer-  
x86_64-linux
```

Check that the file appeared:

```
ls -lh nix-installer-x86_64-linux
```

You should see something like:

```
-rw-r--r-- ... nix-installer-x86_64-linux
```

Then:

```
chmod +x nix-installer-x86_64-linux
```

Then install:

```
./nix-installer-x86_64-linux install \  
--no-confirm \  
--extra-conf "extra-substituters = https://nix-cache.fossi-foundation.org" \  
--extra-conf "extra-trusted-public-keys = nix-cache.fossi-  
foundation.org:3+K59iFwXqKsL7BNu6Guy0v+uTlwsxYQxjspXzqLYQs=" \  
--extra-conf "extra-experimental-features = nix-command flakes"
```

After it finishes:

```
source ~/.profile  
nix -version
```

Load Nix:

```
. /nix/var/nix/profiles/default/etc/profile.d/nix-daemon.sh
```

Verify:

```
nix -version
```

You should see a version number.(Nix installed successfully)

Clone LibreLane:

```
git clone https://github.com/librelane/librelane.git
```

```
cd librelane
```

Enter the development environment:

```
nix develop or use nix-shell
```

Verify:

```
librelane --version
```

```
librelane --smoke-test
```

For Windows (WSL2), begin with installing WSL:

Enter this wsl --install -d Ubuntu on power shell

NB: After Ubuntu launches, follow exactly the same Linux installation procedure (install prerequisites, install Nix, clone LibreLane, run nix develop)

For macOS, include both Intel and Apple Silicon.

Install Apple's Command Line Tools:

```
xcode-select --install
```

Install Homebrew if needed:

```
/bin/bash -c "$(curl -fsSL  
https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

Install Git and Curl if required:

```
brew install git curl
```

Install Nix using the same installer as Linux, then:

```
git clone https://github.com/librelane/librelane.git
```

```
cd librelane
```

```
nix develop or use nix-shell
```

Verify:

```
librelane --version
```

LibreLane supports native execution on both Intel and Apple Silicon Macs through the Nix workflow.

Step 3 · Install your hardened M2 RTL

Copy your Module 2 `elevator.v` (the one that passed the fault-injection harness) into `src/elevator.v`. Do NOT bring forward Module 1 bugs — M3 assumes those fixes exist.

```
cp ../silicon-dreams-m2/src/elevator.v src/elevator.v
git add src/elevator.v
git commit -m "Install M2-hardened elevator RTL"
```

E8873C

If you bring forward a broken Floor B2 reset or a 2-bit state encoding, the integration smoke test will fail BEFORE you even reach the LibreLane flow — don't waste your first run. Confirm M2 smoke passes locally first.

Step 4 · First hardening pass

```
./scripts/run-librelane.sh

# Or for the explicit form:
librelane run \
  --config librelane/config.json \
  --pdk sky130A \
  --scl sky130_fd_sc_hd \
  --tag first-run \
  src/
```

Expected flow stages (in order). Each should tick green:

Synthesis (Yosys)	Runs `synth -flatten`. Emits gate count, longest path estimate.
Floorplanning	Creates the die outline, pad ring, and power grid.
Placement (OpenROAD)	Global + detailed placement of standard cells. Target density ~0.55-0.60.
Clock-Tree Synthesis	Builds the clock tree; inserts buffers; balances skew.
Routing (OpenROAD)	Global + detailed routing. Watch for congestion warnings.
DRC (Magic + KLayout)	Design rule checks against SKY130A. MUST be zero violations.
LVS (Netgen)	Layout-vs-schematic. MUST match.
Timing (OpenSTA)	Setup + hold against corners. MUST meet CLOCK_PERIOD.
Antenna (Magic)	Antenna ratio checks. Should be zero.
GDS write	Final GDSII + LEF emitted to `runs/<tag>/final/`.

Step 5 · Where reports live

```
runs/first-run/
```

```

├─ reports/
│   ├── synthesis/1-synth.stat    # gate count, area, longest path
│   ├── placement/              # density, wirelength
│   ├── cts/                    # clock tree skew, latency
│   ├── routing/                # congestion, shorts
│   ├── signoff/
│   │   ├── drc.rpt             # <-- your eyes live here
│   │   ├── lvs.rpt
│   │   ├── sta-best.rpt
│   │   └── sta-worst.rpt
│   └── metrics.csv             # one-line summary for CI parsing
└─ final/
    ├── gds/top.gds
    ├── lef/top.lef
    └── def/top.def

```

If any stage fails, it will be obvious — the run exits non-zero and the last report file written tells you where. Do not panic. First failures are expected; hardening is iterative.

Troubleshooting

`LibreLane version mismatch (2025.06 != 2025.04)`	You pulled the wrong image. Explicitly pull `efabless/librelane:2025.04` and re-run.
`PDK sky130A not found`	First-time setup needs `./scripts/install-pdk.sh`. Takes ~20 min and ~4 GB disk.
Synthesis times out	Your elevator RTL has a combinational loop or infinite-depth always block. Run M2 smoke again.
Routing emits 100+ shorts	PL_TARGET_DENSITY_PCT is too high. Lower from 0.60 to 0.55 in config.json.
Magic exits on segfault	Magic needs Xvfb in headless. The run script wraps this. If running manually, prefix with `xvfb-run`.

Reflection

- Write down your first-run total time and the stage(s) that crashed or warned. You will compare this to your final-run timing.
- Before moving to SG-M3-02, confirm: synthesis ticks, DRC may fail, LVS may fail, but the flow completes all stages. A flow that dies at placement means your config needs attention first.

6B2E8F

The first LibreLane run is not supposed to be clean. It is a calibration — you are learning which knobs move which reports. Don't try to fix DRC on run 1. That is run 6.